



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico 1: TAD AirMiles.

Especificación del TAD AirMiles: sistema de fidelización de una aerolínea.

13 de septiembre de 2026

Algoritmos y Estructuras de Datos

Los4Bagueados

Integrante	LU	Correo electrónico
Campos, Felipe	356/20	felippe.1114@gmail.com
Portaluppi, Alejandro	51/19	alejandro.portaluppi@outlook.com
Chopite, Julio	617/24	juliodavidchopite@gmail.com
Rychter, Ariel	1294/21	arirychter@gmail.com



Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (+54 +11) 4576-3300

<http://www.exactas.uba.ar>

1. Decisiones, Aclaraciones y Desarrollo.

Al pensar el problema por primera vez, nos dimos cuenta de que podíamos tratar todo como una transacción. Partimos escribiendo dos observadores: uno donde se cargaban las promociones y otro que era un superobservador que consideraba toda operación como una transacción (Transferencias, Acumulaciones, Canjes). Luego tuvimos que rediseñar los observadores, modularizando entre los diferentes tipos de transacciones, principalmente para las acumulaciones, donde teníamos que conectar también el observador de promociones. Esto nos llevó a terminar modularizando todo en cinco observadores para trabajar de forma prolija.

Estos observadores cubren todos los casos. Con el superobservador anterior no se podían cubrir correctamente, principalmente, los procedimientos `acumularMillas` y `resumenTransacciones`. Estos observadores permiten realizar todas las operaciones de forma prolija y sin necesidad de hacer aclaraciones especiales.

Finalmente, los observadores que consideramos minimales para especificar cada proc del TAD quedan de la siguiente manera:

Aclaremos los tipos dado que el struct (`<>`) y las tuplas (`<>`) pueden verse de la misma manera ya que en las macros tienen el mismo símbolo, para que quede claro también agregamos la palabra `tupla` para que puedan diferenciarse a la vista.

El observador `promociones` es un diccionario que tiene como clave al nombre de la promoción (`nombrePromo`) y como valor `parametrosPromo`, que es un struct que tiene un factor y un tope.

El observador `historialDeTransferencias` es una secuencia de transacciones, donde las transacciones son un struct que tiene un origen (quien envía la transferencia), un destino (quien la recibe) y la cantidad a transferir.

El observador `historialDeAcumulaciones` es un diccionario que tiene como clave a los usuarios y como valor una secuencia de acumulaciones. Una acumulación es un struct que contiene la distancia `Volada`, la promoción con la que se vuela, que es una tupla (`nombrePromo`, `factor`) donde `factor` es el factor de promoción, y las `millasGanadas` que se calculan mediante la fórmula dada por el enunciado:

$$millasGanadas = millasVoladas \cdot factorCategoria \cdot factorPromocion$$

El observador `historialDeCanjes` es un diccionario cuyas claves son usuarios y cuyo valor es una secuencia de enteros que representa las millas que se canjean en cada operación de canje que realiza el usuario.

El observador `referidos` es un diccionario que toma como clave a un usuario y como valor una secuencia de enteros con el elemento 500, correspondientes a las millas ganadas por haber referido a alguien (donde tanto el referido como el referidor ganan sus 500 millas). No nos importa a quién refirió, ni por quién fue referido, sino únicamente el valor que ganó para poder contabilizarlo en `millasTotales` y `millasDisponibles`, y también para poder excluirlo en `resumenTransacciones`, que excluye las millas ganadas por referidos.

En el `asegura` del proc `registrarUsuario` hicimos un predicado `seAgregoUsuario(s,u)`, donde tomamos la decisión de asegurarnos de que todos los usuarios queden registrados en los observadores, inicializando los diccionarios con la clave del nuevo usuario y su valor como una secuencia vacía.

El historial de transacciones del sistema es global y no discrimina por usuario, por lo tanto, no se modifica al registrar un usuario. Lo mismo sucede con las promociones, ya que no tienen relación con el registro de un usuario. Esta aclaración la hacemos dado que el enunciado pide explícitamente que las transacciones del usuario se inicien vacías, pero nuestras transacciones no están discriminadas por un usuario particular, sino que contemplan a los dos involucrados.

Por lo tanto, no dependen de un único usuario y no pueden inicializarse como vacías para cada usuario, dado que las planteamos de manera diferente.

Decidimos no hacer explícito los resultados que deben cumplir los auxiliares `millasTotales` y `millasDisponibles`, así como también por ejemplo los topes de la promo, ya que por construcción deberían devolver el resultado correcto y sería redundante volver a ponerlo, si bien ayuda a que quede claro, predicamos sobre los observadores, no sobre el comportamiento de un auxiliar (salvo que sea necesario). Por lo tanto quitamos predicados sobre auxiliares como por ejemplo que al registrarse con referidor, `millasDisponibles` y `millasTotales` deben dar 500 para el nuevo usuario, y sumar 500 a lo que ya tenía el referidor, dejando solamente los predicados sobre observadores (a excepción de que sea necesario el auxiliar, así con cada operación que cambie el resultado de esos y otros auxiliares como puede ser una transferencia, un canje, una acumulación, un referido, el descuento en el tope de una promo, etc).

En esta línea, el enunciado dice que en todo momento debe garantizarse que las millas disponibles de un usuario no superen sus millas totales, pero por construcción, $\text{millasDisponibles} = \text{millasTotales} - \text{millasPerdidas}$, por lo tanto siempre ocurre que $\text{millasDisponibles} \leq \text{millasTotales}$, entonces no hace falta estar verificando continuamente predicados sobre auxiliares dado que están contruídos de forma tal que se respeten las consignas del enunciado. Por otro lado, para cumplir que las operaciones que modifican el sistema deben preservar la integridad de los saldos (ningún usuario puede tener saldo negativo), verificamos que las `millasDisponibles` sean suficientes para realizar la transferencia o canje. Para que se respete que los identificadores de usuario sean números enteros positivos se requieren positivos para ser válidos como entrada, entre otros casos. De esta manera nos aseguramos respetar estos puntos por construcción.

Hay un error en el enunciado de acumularMillas: si un usuario tiene millas totales mayores o iguales a 50.000, se considera de categoría Oro con su factor asociado, y si un usuario tiene millas totales mayores o iguales a 1.000.000, es de categoría Platino. Ahora bien, si un usuario supera o iguala las 1.000.000 millas totales, el usuario es Oro y Platino al mismo tiempo, lo cual sabemos que no es posible, ya que un usuario no puede estar en dos categorías diferentes al mismo tiempo. Tomamos la decisión de resolver este inconveniente del enunciado imponiendo que el factor Oro para el usuario sea válido cuando tenga menos de 1.000.000 de millas totales.

En transferirMillas encontramos un error de enunciado: dos usuarios pueden transferirse entre ellos infinitamente la misma cantidad una y otra vez, y ambos llegar a la categoría Platino. Por recomendación docente, decidimos seguir la directiva del enunciado, permitir el bug y dejar que el programador decida cómo afrontar este error.

En cuanto al tope de una promo, decidimos hacer lo siguiente: tener un tope igual a -1 significa que no hay tope. Esto lo podemos utilizar, por ejemplo, cuando creamos el sistema con el nombre de la promoción No Promo, o cuando creamos una promoción sin tope. El tope es un número que va bajando a medida que los usuarios vuelan con la promoción y acumulan millas, haciendo: $\text{tope} - \text{acumuladoGlobalDeMillasConEsaPromo}$ hasta llegar a 0. Mientras baja el tope, la promoción está activa y, cuando llega a 0, se elimina del sistema. Tomamos la decisión de validar en el requiere que las millas a volar no superen el tope de la promoción: si el vuelo supera el tope de la promoción, no puede realizarse con esa promoción.

Al final del proc `acumularMillas`, se utiliza el predicado `miIfThenElse`, un `IfThenElse` propio que se redefinió para no abusar del `ifThenElse` "nativo" del lenguaje sin saber lo que significa, y demostrar que no estamos usando el `ifThenElse` como si estuviéramos programando, sino el equivalente en su forma lógica. Esto nos permite hacer un `else if` (o `elif`) de forma justificada. Toma tres predicados de tipo booleano y devuelve un valor de verdad. Es muy útil como herramienta, por ejemplo, en `acumularMillas`, donde lo usamos para determinar en qué situación estamos: si el usuario gastó el tope de la promoción, si no llegó al tope o si

la promoción no tiene tope. Con el fin de que quede clara la estructura, el último predicado (*modificacionSegunTope*) que utiliza el *miIfThenElse* propio es equivalente a:

```

if (LaPromoNoTieneTope)
  then casoNoTope
  else if (usuarioGastaElTopeDeLaPromo)
    then casoGastaElTopeDeLaPromo
    else casoNoGastaElTopeDeLaPromo

```

2. Especificacion del TAD AirMiles

Trans ES $\langle origen : \mathbb{Z}, destino : \mathbb{Z}, cantidad : \mathbb{Z} \rangle$

Acumulacion ES $\langle distanciaVolada : \mathbb{R}, promo : tupla \langle nombrePromo, factor \rangle, millasGanadas : \mathbb{R} \rangle$

parametrosPromo ES $\langle factor : \mathbb{R}, tope : \mathbb{R} \rangle$

nombrePromo ES string

factor ES $\mathbb{R}$

Usuario ES $\mathbb{Z}$

Categoria ES *enum* $\{ 'Bronce', 'Plata', 'Oro', 'Platino' \}$

TAD AirMiles {

obs promociones : dict $\langle nombrePromo, parametrosPromo \rangle$

obs historialDeTransferencias : seq $\langle Trans \rangle$

obs historialDeAcumulaciones : dict $\langle Usuario, seq \langle Acumulacion \rangle \rangle$

obs historialDeCanjes : dict $\langle Usuario, seq \langle millas : \mathbb{R} \rangle \rangle$

obs referidos : dict $\langle Usuario, seq \langle cantidad : \mathbb{Z} \rangle \rangle$

aux usuarios($s : AirMiles$) : conj $\langle Usuario \rangle = claves(s.historialDeAcumulaciones)$

aux millasTotales($s : AirMiles, u : Usuario$) : $\mathbb{R} =$
 $sumaDeAcumulaciones(s, u) + sumaDeTransferenciasRecibidas(s, u) + sumaReferidos(s, u)$

aux millasDisponibles($s : AirMiles, u : Usuario$) : $\mathbb{R} =$
 $millasTotales(s, u) - sumaDeCanjes(u, s) - sumaDeTransferenciasEnviadas(s, u)$

aux sumaDeAcumulaciones($s : AirMiles, u : Usuario$) : $\mathbb{R} =$
 $\sum_{i=0}^{|s.historialDeAcumulaciones[u]|-1} s.historialDeAcumulaciones[u][i].millasGanadas$

aux sumaDeTransferenciasRecibidas($s : AirMiles, u : Usuario$) : $\mathbb{R} =$
 $\sum_{i=0}^{|s.historialDeTransferencias|-1} \text{IfThenElse}(s.historialDeTransferencias[i].destino =$
 $u, s.historialDeTransferencias[i].cantidad, 0)$

aux sumaDeTransferenciasEnviadas($s : AirMiles, u : Usuario$) : $\mathbb{R} =$
 $\sum_{i=0}^{|s.historialDeTransferencias|-1} \text{IfThenElse}(s.historialDeTransferencias[i].origen =$
 $u, s.historialDeTransferencias[i].cantidad, 0)$

aux sumaReferidos($s : AirMiles, u : Usuario$) : $\mathbb{R} = \sum_{i=0}^{|s.referidos[u]|-1} s.referidos[u][i]$

aux sumaDeCanjes($u : Usuario, s : AirMiles$) : $\mathbb{R} =$

$$\sum_{i=0}^{|s.historialDeCanjes[u]|-1} s.historialDeCanjes[u][i]$$

```

aux categoria(s : AirMiles, u : Usuario) : Categoria =
  (res = "Bronce"  $\iff$  (millasTotales(s, u) < 10,000))  $\wedge$ 
  (res = "Plata"  $\iff$  10,000  $\leq$  millasTotales(s, u) < 50,000)  $\wedge$ 
  (res = "Oro"  $\iff$  50,000  $\leq$  millasTotales(s, u) < 1,000,000)  $\wedge$ 
  (res = "Platino"  $\iff$  millasTotales(s, u)  $\geq$  1,000,000)

aux factorCategoria(s : AirMiles, u : Usuario) :  $\mathbb{R}$  =
  (res = 1  $\iff$  categoria(s, u) = "Bronce")  $\wedge$ 
  (res = 1,5  $\iff$  categoria(s, u) = "Plata")  $\wedge$ 
  (res = 2  $\iff$  categoria(s, u) = "Oro")  $\wedge$ 
  (res = 0  $\iff$  categoria(s, u) = "Platino")

proc crearSistema() : AirMiles {
  requiere { True }
  asegura {
    res.historialDeAcumulaciones = {}  $\wedge$ 
    res.historialDeTransferencias = <>  $\wedge$ 
    res.historialDeCanjes = {}  $\wedge$ 
    res.referidos = {}  $\wedge$ 
    res.promociones = {"NoPromo" : (1, -1)}
  }
}

proc registrarUsuario(inout s : AirMiles, in u : Usuario) {
  requiere { u > 0  $\wedge$   $\neg$ usuarioEnSistema(s, u)  $\wedge$  s = S0 }
  asegura {
    seAgregoUsuario(s, u)  $\wedge$ 
    lasDemasAcumulacionesNoCambian(s, S0, u)  $\wedge$ 
    losDemasCanjesNoCambian(s, S0, u)  $\wedge$ 
    losDemasReferidosNoCambian(s, S0, u)  $\wedge$ 
    transferenciasNoCambian(s, S0)  $\wedge$ 
    promocionesNoCambia(s, S0)  $\wedge$ 
    millasDisponibles(s, u) = 0  $\wedge$  millasTotales(s, u) = 0
  }
}

pred usuarioEnSistema(s : AirMiles, u : Usuario) {
  u  $\in$  usuarios(s)
}

pred seAgregoUsuario(s : AirMiles, u : Usuario) {
  u  $\in$  s.historialDeAcumulaciones  $\wedge$  s.historialDeAcumulaciones[u] = <>  $\wedge$ 
  u  $\in$  s.historialDeCanjes  $\wedge$  s.historialDeCanjes[u] = <>  $\wedge$ 
  u  $\in$  s.referidos  $\wedge$  s.referidos[u] = <>
}

pred lasDemasAcumulacionesNoCambian(s : AirMiles, S0 : AirMiles, u : Usuario) {

```

```

    (∀usu : Usuario)(usu ∈ claves(S0.historialDeAcumulaciones) →
    usu ∈ claves(s.historialDeAcumulaciones)) ∧
    (∀usu : Usuario)((usu ∈ claves(S0.historialDeAcumulaciones) ∧ usu ≠ u) →L
    s.historialDeAcumulaciones[usu] = S0.historialDeAcumulaciones[usu])
}

pred losDemasCanjesNoCambian(s : AirMiles, S0 : AirMiles, u : Usuario) {
    (∀usu : Usuario)(usu ∈ claves(S0.historialDeCanjes) → usu ∈ claves(s.historialDeCanjes)) ∧
    (∀usu : Usuario)((usu ∈ claves(S0.historialDeCanjes) ∧ usu ≠ u) →L
    s.historialDeCanjes[usu] = S0.historialDeCanjes[usu])
}

pred losDemasReferidosNoCambian(s : AirMiles, S0 : AirMiles, u : Usuario) {
    (∀usu : Usuario)(usu ∈ claves(S0.referidos) → usu ∈ claves(s.referidos)) ∧
    (∀usu : Usuario)((usu ∈ claves(S0.referidos) ∧ usu ≠ u) →L s.referidos[usu] =
    S0.referidos[usu])
}

pred transferenciasNoCambian(s : AirMiles, S0 : AirMiles) {
    s.historialDeTransferencias = S0.historialDeTransferencias
}

pred promocionesNoCambia(s : AirMiles, S0 : AirMiles) {
    s.promociones = S0.promociones
}

proc registrarUsuarioConReferidor(inout s : AirMiles, in ref : Usuario, in u : Usuario)
{
    requiere { u > 0 ∧ ¬usuarioEnSistema(s, u) ∧ usuarioEnSistema(s, ref) ∧ s = S0
    }
    asegura {
        seAgregoUsuario(s, u) ∧
        seAgregoReferido(s, S0, u, ref) ∧
        lasDemasAcumulacionesNoCambian(s, S0, u) ∧
        losDemasCanjesNoCambian(s, S0, u) ∧
        transferenciasNoCambian(s, S0) ∧
        losOtrosReferidosNoCambian(s, S0, u, ref) ∧
        promocionesNoCambia(s, S0)
    }
}

pred seAgregoReferido(s : AirMiles, S0 : AirMiles, u : Usuario, ref : Usuario) {
    s.referidos[u] = S0.referidos[u] ++ < 500 > ∧
    s.referidos[ref] = S0.referidos[ref] ++ < 500 >
}

pred losOtrosReferidosNoCambian(s : AirMiles, S0 : AirMiles, u : Usuario, ref :
Usuario) {
    (∀usu : ℤ)(usu ∈ claves(S0.referidos) ∧ → usu ∈ claves(s.referidos)) ∧
    (∀usu : ℤ)((usu ∈ claves(S0.referidos) ∧ usu ≠ u ∧ usu ≠ ref) →L S0.referidos[usu] =
    s.referidos[usu])
}

```

```

}

proc acumularMillas(in u : Usuario, in millasVoladas :  $\mathbb{R}$ , in promo : nombrePromo, inout s :
AirMiles) {
  requiere {
    (s = s0)  $\wedge$  (usuarioEnSistema(S0, u))  $\wedge$ 
    (existePromo(promo, S0))  $\wedge$ 
    (millasVoladas > 0)  $\wedge$ 
    (promoDisponeDeMillasParaVuelo(u, promo, millasVoladas, s0))
  }
  asegura {
    canjesNoCambia(s0, s)  $\wedge$ 
    referidosNoCambia(s0, s)  $\wedge$ 
    transferenciasNoCambian(s0, s)  $\wedge$ 
    esSubSeq(s0.historialDeAcumulaciones[u], s.historialDeAcumulaciones[u])  $\wedge$ 
    modificacionSegunTope(u, millasVoladas, promo, s, s0)
  }
}

pred existePromo(promo : nombrePromo, s : AirMiles) {
  promo  $\in$  claves(s.promociones)
}

pred promoDisponeDeMillasParaVuelo(u : Usuario, promo : nombrePromo, millasAVolar :
 $\mathbb{R}$ , s : AirMiles) {
  s.promociones[promo].tope = -1  $\vee$ 
  millasAVolar * factorCategoria(s, u) * s.promociones[promo].factor  $\leq$  s.promociones[promo].tope
}

pred esSubSeq(S0 : seq⟨T⟩, S : seq⟨T⟩) {
  (|S0|  $\leq$  |S|)
   $\wedge_L (\forall i : \mathbb{Z}) ((0 \leq i < |S_0|) \rightarrow_L (S_0[i] = S[i]))$ 
}

pred miIfThenElse(predicado : bool, Then : bool, Else : bool) {
  (predicado  $\wedge$  Then)  $\vee$  ( $\neg$ predicado  $\wedge$  Else)
}

pred promoNoTieneTope(promo : nombrePromo, s0 : AirMiles) {
  s0.promociones[promo].tope = -1
}

pred usuarioGastaElTopeDeLaPromo(u : Usuario, millasVoladas :  $\mathbb{R}$ , promo : nombrePromo, s0 :
AirMiles) {
  millasVoladas * factorCategoria(s0, u) * s0.promociones[promo].factor =
  s0.promociones[promo].tope
}

pred seAgregoVuelo(vuelo : Acumulacion, s : seq⟨Acumulacion⟩, s0 : seq⟨Acumulacion⟩)
{
  (vuelo  $\in$  s)  $\wedge$  (aparicionesEnSeqDe(vuelo, s) = aparicionesEnSeqDe(vuelo, s0) + 1)
}

```

```

}

pred usuarioTiene1VueloMas( $u : Usuario, s : AirMiles, s_0 : AirMiles$ ) {
   $|s.historialDeAcumulaciones[u]| = |s_0.historialDeAcumulaciones[u]| + 1$ 
}

pred soloCambioLaPromoP( $promo : nombrePromo, s : AirMiles, s_0 : AirMiles$ ) {
  ( $promo \in claves(s.promociones)$ )  $\wedge$ 
  ( $s.promociones[promo] \neq s_0.promociones[promo]$ )  $\wedge$ 
  ( $\forall p : claves(s_0.promociones) ((promo \neq p) \rightarrow_L (s.promociones[p] = s_0.promociones[p]))$ )
}

aux crearAcumulacion( $distancia : \mathbb{R}, NombrePromo : nombrePromo, factor : \mathbb{R}, millasGanadas : \mathbb{R}$ ) :  $Acumulacion =$ 
   $\langle distanciaVolada : distancia,$ 
   $promo : tupla\langle NombrePromo, factor \rangle,$ 
   $millasGanadas : millasGanadas \rangle$ 

pred casoNoTope( $u : Usuario, millasVoladas : \mathbb{R}, promo : nombrePromo, s : AirMiles, s_0 : AirMiles$ ) {
   $promocionesNoCambia(s, s_0) \wedge$ 
   $usuarioTiene1VueloMas(u, s, s_0) \wedge$ 
   $seAgregoVuelo(crearAcumulacion($ 
   $millasVoladas, promo, s_0.promociones[promo].factor, millasVoladas * factorCategoria(s_0, u) *$ 
   $s_0.promociones[promo].factor), s.historialDeAcumulaciones[u], s_0.historialDeAcumulaciones[u])$ 
   $\wedge lasDemasAcumulacionesNoCambian(s, s_0, u)$ 
}

pred casoGastaElTopeDeLaPromo( $u : Usuario, millasVoladas : \mathbb{R}, promo : nombrePromo, s : AirMiles, s_0 : AirMiles$ ) {
   $usuarioTiene1VueloMas(u, s, s_0) \wedge$ 
   $seAgregoVuelo(crearAcumulacion($ 
   $millasVoladas, promo, s_0.promociones[promo].factor, millasVoladas * factorCategoria(s_0, u) *$ 
   $s_0.promociones[promo].factor), s.historialDeAcumulaciones[u], s_0.historialDeAcumulaciones[u])$ 
   $\wedge seEliminaPromocion(s_0, s, promo) \wedge$ 
   $lasDemasAcumulacionesNoCambian(s, s_0, u)$ 
}

pred casoNoGastaElTopeDeLaPromo( $u : Usuario, millasVoladas : \mathbb{R}, promo : nombrePromo, s : AirMiles, s_0 : AirMiles$ ) {
   $usuarioTiene1VueloMas(u, s, s_0) \wedge$ 
   $seAgregoVuelo(crearAcumulacion($ 
   $millasVoladas, promo, s_0.promociones[promo].factor, millasVoladas * factorCategoria(s_0, u) *$ 
   $s_0.promociones[promo].factor), s.historialDeAcumulaciones[u], s_0.historialDeAcumulaciones[u])$ 
   $\wedge s.promociones[promo].tope = s_0.promociones[promo].tope - (millasVoladas * factorCategoria(s_0, u) * s_0.promociones[promo].factor) \wedge$ 
   $|s.promociones| = |s_0.promociones| \wedge$ 
   $soloCambioLaPromoP(promo, s, s_0) \wedge$ 
   $lasDemasAcumulacionesNoCambian(s, s_0, u)$ 
}

```

}

pred *modificacionSegunTope*(*u* : *Usuario*, *millasVoladas* : $\mathbb{R}$, *promo* : *nombrePromo*, *s* : *AirMiles*, *s₀* : *AirMiles*) {

miIfThenElse((*promoNoTieneTope*),
 (*casoNoTope*(*u*, *millasVoladas*, *promo*, *s*, *s₀*)),
 (*miIfThenElse*(
 (*usuarioGastaElTopeDeLaPromo*(*u*, *millasVoladas*, *promo*, *s₀*)),
 (*casoGastaElTopeDeLaPromo*(*u*, *millasVoladas*, *promo*, *s*, *s₀*)),
 (*casoNoGastaElTopeDeLaPromo*(*u*, *millasVoladas*, *promo*, *s*, *s₀*)))

}

proc *canjearMillas*(inout *s* : *AirMiles*, in *u* : *Usuario*, in *millas* : $\mathbb{R}$) {

requiere {
 s = *S₀* $\wedge$ *millas* > 0 $\wedge$ *usuarioEnSistema*(*s*, *u*) $\wedge_L$ *millas* $\leq$ *millasDisponibles*(*s*, *u*)
 }
 asegura {
 realizarCanje(*s*, *S₀*, *u*, *millas*) $\wedge$
 losDemasCanjesNoCambian(*s*, *S₀*, *u*) $\wedge$
 promocionesNoCambia(*s*, *S₀*) $\wedge$
 acumulacionesNoCambia(*s*, *S₀*) $\wedge$
 referidosNoCambia(*s*, *S₀*) $\wedge$
 transferenciasNoCambian(*s*, *S₀*)
 }

}

pred *realizarCanje*(*s* : *AirMiles*, *S₀* : *AirMiles*, *u* : *Usuario*, *millas* : $\mathbb{R}$) {

s.historialDeCanjes[*u*] = *S₀.historialDeCanjes*[*u*] + + < *millas* > $\wedge$
 millasDisponibles(*s*, *u*) = *millasDisponibles*(*S₀*, *u*) - *millas*

}

pred *acumulacionesNoCambia*(*s* : *AirMiles*, *S₀* : *AirMiles*) {

s.historialDeAcumulaciones = *S₀.historialDeAcumulaciones*

}

pred *referidosNoCambia*(*s* : *AirMiles*, *S₀* : *AirMiles*) {

s.referidos = *S₀.referidos*

}

proc *transferirMillas*(in *origen* : $\mathbb{Z}$, in *destino* : $\mathbb{Z}$, in *monto* : $\mathbb{R}$, inout *S* : *AirMiles*) : {

requiere {
 (*s* = *s₀*) $\wedge$ *monto* > 0 $\wedge$ (*usuarioEnSistema*(*s₀*, *origen*)) $\wedge$ (*usuarioEnSistema*(*s₀*, *destino*)) $\wedge$
 (*origen* $\neq$ *destino*) $\wedge$ *disponeDeMillas*(*origen*, *monto*, *s₀*)
 }
 asegura {

```

    (promocionesNoCambia( $s_0, s$ ))  $\wedge$ 
    (acumulacionesNoCambia( $s_0, s$ ))  $\wedge$ 
    (canjesNoCambia( $s_0, s$ ))  $\wedge$ 
    (referidosNoCambia( $s_0, s$ ))  $\wedge$ 
    tamañoNuevoHistorialDeTransSuma1( $s, s_0$ )  $\wedge$ 
    esSubSeq( $s_0.historialDeTransferencias, s.historialDeTransferencias$ )  $\wedge$ 
    seAgregoTrans( $\langle origen : origen, destino : destino, cantidad : monto \rangle,$ 
     $s_0.historialDeTransferencias, s.historialDeTransferencias$ )
  }
}

pred canjesNoCambia( $s : AirMiles, S_0 : AirMiles$ ) {
   $s.historialDeCanjes = S_0.historialDeCanjes$ 
}

pred tamañoNuevoHistorialDeTransSuma1(in  $s : Airmiles, in s_0 : Airmiles$ ) {
   $|s.historialDeTransferencias| = |s_0.historialDeTransferencias| + 1$ 
}

pred seAgregoTrans( $trans : Trans, S_0 : seq\langle Trans \rangle, S : seq\langle Trans \rangle$ ) {
  ( $trans \in S$ )
   $\wedge ((aparicionesEnSeqDe(trans, S_0) + 1 = aparicionesEnSeqDe(trans, S))$ 
}

pred disponeDeMillas( $u : Usuario, millas : \mathbb{R}, s : AirMiles$ ) {
   $millasDisponibles(s, u) \geq millas$ 
}

aux aparicionesEnSeqDe( $e : T, s : seq\langle T \rangle$ ) :  $\mathbb{Z} =$ 
   $\sum_{i=0}^{|s|-1} ifThenElse(s[i] = e, 1, 0)$ 

proc consultarSaldo(in  $s : AirMiles, in u : Usuario$ ) :  $\mathbb{R}$  {
  requiere {  $usuarioEnSistema(s, u)$  }
  asegura {  $res = millasDisponibles(s, u)$  }
}

proc consultarCategoria(in  $s : AirMiles, in u : Usuario$ ) : Categoria {
  requiere {  $usuarioEnSistema(s, u)$  }
  asegura {  $res = categoria(s, u)$  }
}

proc rankingMillas(in  $N : \mathbb{Z}, in s : AirMiles$ ) :  $seq\langle Usuario \rangle$  {
  requiere {
     $N > 0$ 
  }
  asegura {
     $estaOrdenadoSegunCantidadMillasDisponibles(res, s) \wedge$ 
     $todosUsuariosEstanEnElHistorial(res, s) \wedge$ 
     $resTieneTamañoMenorOIgualQueN(res, s, N) \wedge$ 
     $todosUsuariosDistintos(res) \wedge$ 
     $todosLosNoSeleccionadosTienenMenosOMismasMillas(res, s)$ 
  }
}

```

```

    }
}

pred estaOrdenadoSegunCantidadMillasDisponibles(res : seq⟨Usuario⟩, s : AirMiles)
{
    (∀ i : ℤ)(0 ≤ i < |res| - 1 →L millasDisponibles(s, res[i]) ≥ millasDisponibles(s, res[i + 1]))
}

pred todosUsuariosEstanEnElHistorial(res : seq⟨Usuario⟩, s : AirMiles) {
    (∀ i : ℤ)(0 ≤ i < |res| →L res[i] ∈ usuarios(s))
}

pred resTieneTamañoMenorOIgualQueN(res : seq⟨Usuario⟩, s : AirMiles, N : ℤ) {
    (N ≤ |usuarios(s)| ∧ |res| = N) ∨
    (|usuarios(s)| < N ∧ |res| = |usuarios(s)|)
}

pred todosUsuariosDistintos(res : seq⟨Usuario⟩) {
    (∀ i : ℤ)(0 ≤ i < |res| →L (∀ j : ℤ)(0 ≤ j < |res| ∧ i ≠ j →L res[i] ≠ res[j]))
}

pred todosLosNoSeleccionadosTienenMenosOMismasMillas(res : seq⟨Usuario⟩, s : AirMiles)
{
    (∀ u : Usuario)(u ∈ usuarios(s) ∧ u ∉ res ∧ |res| > 0 →L
    millasDisponibles(s, u) ≤ millasDisponibles(s, res[|res| - 1]))
}

proc resumenTransacciones(in u : Usuario, in s : AirMiles) : ⟨total : ℝ, canjeadas :
ℝ, enviadas : ℝ, recibidas : ℝ⟩ {
    requiere { usuarioEnSistema(s, u) }
    asegura {
        (res.total = millasTotales(s, u)) ∧
        (res.canjeadas = sumaDeCanjes(u, s)) ∧
        (res.enviadas = sumaDeTransferenciasEnviadas(s, u)) ∧
        (res.recibidas = sumaDeTransferenciasRecibidas(s, u))
    }
}

proc iniciarPromocion(inout s : AirMiles, in nombre : seq⟨char⟩, in factor : ℝ) {
    requiere {
        |nombre| > 0 ∧
        factor ≥ 1 ∧
        s = s0 ∧
        ¬existePromo(nombre, s0)
    }
    asegura {
        seAgregaPromocion(s0, s, nombre, ⟨factor : factor, tope : -1, 0⟩) ∧
        acumulacionesNoCambia(s0, s) ∧
        canjesNoCambia(s0, s) ∧
        transferenciasNoCambian(s0, s) ∧
        referidosNoCambia(s0, s)
    }
}

```

```

    }
}

proc iniciarPromocionConTope(inout s : AirMiles, in nombre : seq⟨char⟩, in factor : ℝ, in tope :
ℝ) {
  requiere {
    |nombre| > 0 ∧
    factor ≥ 1 ∧
    tope > 0 ∧
    s = s0 ∧
    ¬existePromo(nombre, s0)
  }
  asegura {
    seAgregaPromocion(s0, s, nombre, ⟨factor : factor, tope : tope⟩) ∧
    acumulacionesNoCambia(s0, s) ∧
    canjesNoCambia(s0, s) ∧
    transferenciasNoCambian(s0, s) ∧
    referidosNoCambia(s0, s)
  }
}

proc finalizarPromocion(inout s : AirMiles, in nombre : seq⟨char⟩) {
  requiere {
    |nombre| > 0 ∧
    nombre ≠ "NoPromo" ∧
    nombre ∈ s.promociones ∧
    s = s0
  }
  asegura {
    seEliminaPromocion(s0, s, nombre) ∧
    acumulacionesNoCambia(s0, s) ∧
    canjesNoCambia(s0, s) ∧
    transferenciasNoCambian(s0, s) ∧
    referidosNoCambia(s0, s)
  }
}

pred seAgregaPromocion(s0 : AirMiles, s : AirMiles, nombre : seq⟨char⟩, datosPromo :
parametrosPromo) {
  |s.promociones| = |s0.promociones| + 1 ∧
  nombre ∈ s.promociones ∧
  s.promociones[nombre] = datosPromo ∧
  (∀r : seq⟨char⟩)(r ∈ s0.promociones →L
    (r ∈ s.promociones ∧L s.promociones[r] = s0.promociones[r]))
}

pred seEliminaPromocion(s0 : AirMiles, s : AirMiles, nombre : seq⟨char⟩) {

```

```

    |s0.promociones| = |s.promociones| + 1 ∧
    nombre ∉ s.promociones ∧
    (∀r : seq⟨char⟩)(r ∈ s.promociones →L
        (r ∈ s0.promociones ∧L s0.promociones[r] = s.promociones[r]))
}

proc elQueTieneMasMillas(in s : AirMiles) : Usuario {
    requiere {
        |usuarios(s)| > 0
    }
    asegura {
        usuarioEnSistema(s, res) ∧L
        tieneMasMillasQueLosDemas(res, s)
    }
}

pred tieneMasMillasQueLosDemas(res : Usuario, s : AirMiles) {
    (∀u : Usuario)(u ∈ usuarios(s) →L millasTotales(s, u) ≤ millasTotales(s, res))
}

proc elMasCanjeador(in s : AirMiles) : Usuario {
    requiere {
        |usuarios(s)| > 0
    }
    asegura {
        res ∈ usuarios(s) ∧L (∀k : Usuario)(k ∈
            usuarios(s) →L (millasPerdidas(k, s) ≤ millasPerdidas(res, s)))
    }
}

aux millasPerdidas(u : Usuario, s : AirMiles) : ℝ =
    sumaDeCanjes(u, s) + sumaDeTransferenciasEnviadas(s, u)

proc paresTransferidoresExclusivos(in s : AirMiles) : res : seq⟨tupla⟨Usuario, Usuario⟩⟩
{
    requiere {
        True
    }
    asegura {
        todosLosParesEnResSonExclusivos(res, s) ∧
        enResEstanTodosLosExclusivos(res, s) ∧
        noHayParesRepetidos(res) ∧
        noSeInviertenEnRes(res)
    }
}

pred sonTransferidores(a : Usuario, b : Usuario, s : AirMiles) {
    (∃i : ℤ)(0 ≤ i < |s.historialDeTransferencias| ∧
        ((s.historialDeTransferencias[i].origen = a ∧
            s.historialDeTransferencias[i].destino = b) ∨

```

```

    (s.historialDeTransferencias[i].origen = b ∧
     s.historialDeTransferencias[i].destino = a)))
  }

pred usuarioAsoloLeTransfiereExclusivamenteAlUsuarioB(a : Usuario, b : Usuario, s :
AirMiles) {
  (∀ i : ℤ)(0 ≤ i < |s.historialDeTransferencias| →L
  (s.historialDeTransferencias[i].origen = a → s.historialDeTransferencias[i].destino =
  b))
}

pred sonTransferidoresExclusivos(a : Usuario, b : Usuario, s : AirMiles) {
  sonTransferidores(a, b, s) ∧
  usuarioAsoloLeTransfiereExclusivamenteAlUsuarioB(a, b, s) ∧
  usuarioAsoloLeTransfiereExclusivamenteAlUsuarioB(b, a, s)
}

pred todosLosParesEnResSonExclusivos(res : seq⟨tupla⟨Usuario, Usuario⟩⟩, s : AirMiles)
{
  (∀ i : ℤ)(0 ≤ i < |res| →L sonTransferidoresExclusivos(res[i]0, res[i]1, s))
}

pred enResEstanTodosLosExclusivos(res : seq⟨tupla⟨Usuario, Usuario⟩⟩, s : AirMiles)
{
  (∀ a, b : Usuario)((usuarioEnSistema(s, a) ∧ usuarioEnSistema(s, b)) ∧L
  sonTransferidoresExclusivos(a, b, s)) →L (tupla⟨a, b⟩ ∈ res ∨ tupla⟨b, a⟩ ∈ res))
}

pred noHayParesRepetidos(res : seq⟨tupla⟨Usuario, Usuario⟩⟩) {
  (∀ i, j : ℤ)((0 ≤ i < |res| ∧ 0 ≤ j < |res| ∧ i ≠ j) →L res[i] ≠ res[j])
}

pred noSeInviertenEnRes(res : seq⟨tupla⟨Usuario, Usuario⟩⟩) {
  (∀ i : ℤ)(0 ≤ i < |res| →L tupla⟨res[i]1, res[i]0⟩ ∉ res)
}
}

```